

COMP111: Artificial Intelligence

More on uninformed tree search

Frank Wolter

Recap

Basic uninformed tree search algorithms:

- ▶ **Breadth-first search (BFS)**: expand the **oldest** path in the frontier (FIFO queue). Complete and optimal, but memory hungry.

Recap

Basic uninformed tree search algorithms:

- ▶ **Breadth-first search (BFS)**: expand the **oldest** path in the frontier (FIFO queue). Complete and optimal, but memory hungry.
- ▶ **Depth-first search (DFS)**: expand the **newest** path in the frontier (LIFO stack). Very modest memory use, but incomplete (it can get lost down an infinite branch) and not optimal.

Terminology and notation (used from now on)

- ▶ $b =$ **branching factor**: maximum number of successors of a state.
- ▶ $d =$ **depth of the shallowest goal**: length of a shortest path from the start state to a goal state.
- ▶ $m =$ **maximum depth** of the search tree ($m = \infty$ if paths can be arbitrarily long).
- ▶ $\ell =$ **depth limit** that we impose (introduced below).

Terminology and notation (used from now on)

- ▶ b = **branching factor**: maximum number of successors of a state.
- ▶ d = **depth of the shallowest goal**: length of a shortest path from the start state to a goal state.
- ▶ m = **maximum depth** of the search tree ($m = \infty$ if paths can be arbitrarily long).
- ▶ ℓ = **depth limit** that we impose (introduced below).
- ▶ A path is **generated** when it is put on the frontier; it is **expanded** when it is taken off the frontier and its successors are generated.
- ▶ **Complete**: if a solution exists, the algorithm finds one.
- ▶ **Optimal**: the solution returned is a shortest path.
- ▶ **Time complexity**: number of paths generated. **Space complexity**: maximum size of the frontier.

Recap: BFS and DFS side by side

	Complete?	Optimal?	Time	Space
BFS	yes	yes	$O(b^d)$	$O(b^d)$
DFS	no	no	$O(b^m)$	$O(bm)$

The whole of this section is about the gap in this table: BFS has the **good guarantees**, DFS has the **good memory behaviour**. Can we have both?

Overview

- ▶ Variations and combinations:
 - ▶ Depth limited search (DLS)
 - ▶ Iterative deepening search (IDS)
- ▶ Speeding up techniques:
 - ▶ Avoiding repeated states
 - ▶ Bi-directional search

Overview

- ▶ Variations and combinations:
 - ▶ Depth limited search (DLS)
 - ▶ Iterative deepening search (IDS)
- ▶ Speeding up techniques:
 - ▶ Avoiding repeated states
 - ▶ Bi-directional search
- ▶ Where these ideas are still used today (chess engines, Rubik's cube solvers, planners).

Depth Limited Search (DLS)

- ▶ Depth-first search has one very desirable property: space complexity.

Depth Limited Search (DLS)

- ▶ Depth-first search has one very desirable property: space complexity.
- ▶ But if a wrong path is expanded (with no solution on it), then the search may take a very long time, or may not terminate at all.

Depth Limited Search (DLS)

- ▶ Depth-first search has one very desirable property: space complexity.
- ▶ But if a wrong path is expanded (with no solution on it), then the search may take a very long time, or may not terminate at all.
- ▶ Idea: fix a **depth limit** ℓ and never expand a path of length ℓ .

Depth Limited Search (DLS)

- ▶ Depth-first search has one very desirable property: space complexity.
- ▶ But if a wrong path is expanded (with no solution on it), then the search may take a very long time, or may not terminate at all.
- ▶ Idea: fix a **depth limit** ℓ and never expand a path of length ℓ .
- ▶ So DLS = DFS + “stop going deeper at level ℓ ”.

Depth Limited Search (DLS)

- ▶ Depth-first search has one very desirable property: space complexity.
- ▶ But if a wrong path is expanded (with no solution on it), then the search may take a very long time, or may not terminate at all.
- ▶ Idea: fix a **depth limit** ℓ and never expand a path of length ℓ .
- ▶ So DLS = DFS + “stop going deeper at level ℓ ”.
- ▶ Useful if you know (or can bound) the length of a shortest solution.

Depth Limited Search (DLS)

- 1: **Input:** a start state s_0
- 2: for each state s the successors of s
- 3: a test $\text{goal}(s)$ checking whether s is a goal state
- 4: non-negative integer DepthLimit
- 5:
- 6: Set $\text{frontier} := \{s_0\}$
- 7: **while** frontier is not empty **do**
- 8: select and remove from frontier the path $s_0 \dots s_k$ added
- 9: last to frontier
- 10: **if** $\text{goal}(s_k)$ **then**
- 11: return $s_0 \dots s_k$ (and terminate)
- 12: **else** If $k < \text{DepthLimit}$, then for every successor s of s_k
- 13: add $s_0 \dots s_k s$ to frontier
- 14: **end if**
- 15: **end while**

Choosing the depth limit: Romania

- ▶ Only 20 cities on the map $\implies$ no path without repeated states is longer than 19. So $\ell = 19$ is **safe**.

Choosing the depth limit: Romania

- ▶ Only 20 cities on the map $\implies$ no path without repeated states is longer than 19. So $\ell = 19$ is **safe**.
- ▶ Careful examination shows that any city can be reached from any other in at most 10 steps. So also $\ell = 10$ is safe.

Choosing the depth limit: Romania

- ▶ Only 20 cities on the map $\implies$ no path without repeated states is longer than 19. So $\ell = 19$ is **safe**.
- ▶ Careful examination shows that any city can be reached from any other in at most 10 steps. So also $\ell = 10$ is safe.
- ▶ In general: a safe depth limit is any upper bound on the longest shortest path between two states.

Example: a safe depth limit for Rubik's Cube

Recall from Section 2: the cube has 43,252,003,274,489,856,000 states. So how deep could a solution possibly be?

Example: a safe depth limit for Rubik's Cube

Recall from Section 2: the cube has 43,252,003,274,489,856,000 states. So how deep could a solution possibly be?

- ▶ In 2010 it was proved (by exhaustive computer search) that **every** configuration can be solved in at most **20** moves. (This number is sometimes called God's number.)

Example: a safe depth limit for Rubik's Cube

Recall from Section 2: the cube has 43,252,003,274,489,856,000 states. So how deep could a solution possibly be?

- ▶ In 2010 it was proved (by exhaustive computer search) that **every** configuration can be solved in at most **20** moves. (This number is sometimes called God's number.)
- ▶ Hence $\ell = 20$ is a *provably safe* depth limit — and DLS with $\ell = 20$ is complete for the cube, even though plain DFS could wander forever.

Example: a safe depth limit for Rubik's Cube

Recall from Section 2: the cube has 43,252,003,274,489,856,000 states. So how deep could a solution possibly be?

- ▶ In 2010 it was proved (by exhaustive computer search) that **every** configuration can be solved in at most **20** moves. (This number is sometimes called God's number.)
- ▶ Hence $\ell = 20$ is a *provably safe* depth limit — and DLS with $\ell = 20$ is complete for the cube, even though plain DFS could wander forever.
- ▶ The state space is enormous, but it is **shallow**: $b \approx 18$, $d \leq 20$. This is exactly the regime in which depth-limited and iterative-deepening methods are good options.

Example: a safe depth limit for Rubik's Cube

Recall from Section 2: the cube has 43,252,003,274,489,856,000 states. So how deep could a solution possibly be?

- ▶ In 2010 it was proved (by exhaustive computer search) that **every** configuration can be solved in at most **20** moves. (This number is sometimes called God's number.)
- ▶ Hence $\ell = 20$ is a *provably safe* depth limit — and DLS with $\ell = 20$ is complete for the cube, even though plain DFS could wander forever.
- ▶ The state space is enormous, but it is **shallow**: $b \approx 18$, $d \leq 20$. This is exactly the regime in which depth-limited and iterative-deepening methods are good options.

Moral: knowing a bound on d can be worth more than knowing anything about the size of the state space.

Properties of Depth Limited Search (DLS)

- ▶ **Not complete:** if $d > \ell$ then DLS returns failure although a solution exists. If $d \leq \ell$, DLS does find a solution.

Properties of Depth Limited Search (DLS)

- ▶ **Not complete:** if $d > \ell$ then DLS returns failure although a solution exists. If $d \leq \ell$, DLS does find a solution.
- ▶ **Not optimal:** it returns the first solution it stumbles on, which need not be a shortest one.

Properties of Depth Limited Search (DLS)

- ▶ **Not complete:** if $d > \ell$ then DLS returns failure although a solution exists. If $d \leq \ell$, DLS does find a solution.
- ▶ **Not optimal:** it returns the first solution it stumbles on, which need not be a shortest one.
- ▶ **Time:** in the worst case DLS generates

$$1 + b + b^2 + \cdots + b^\ell = O(b^\ell)$$

paths.

Properties of Depth Limited Search (DLS)

- ▶ **Not complete:** if $d > \ell$ then DLS returns failure although a solution exists. If $d \leq \ell$, DLS does find a solution.
- ▶ **Not optimal:** it returns the first solution it stumbles on, which need not be a shortest one.
- ▶ **Time:** in the worst case DLS generates

$$1 + b + b^2 + \cdots + b^\ell = O(b^\ell)$$

paths.

- ▶ **Space:** the frontier holds at most b siblings for each of the ℓ levels of the current branch, so at most $b \cdot \ell$ paths, i.e. $O(b\ell)$.

Quiz (2 minutes, talk to your neighbour)

A state space has $b = 4$ and a shortest solution at depth $d = 6$.
You run DLS with limit $\ell = 4$.

1. What does the algorithm return?
2. Does it terminate?
3. You now rerun it with $\ell = 8$. Is the returned solution of length 6?

Quiz (2 minutes, talk to your neighbour)

A state space has $b = 4$ and a shortest solution at depth $d = 6$.
You run DLS with limit $\ell = 4$.

1. What does the algorithm return?
2. Does it terminate?
3. You now rerun it with $\ell = 8$. Is the returned solution of length 6?

Answers: (1) failure — no solution has length ≤ 4 ; (2) yes, always: the tree below depth ℓ is finite; (3) not necessarily — DLS is not optimal, it may return a solution of length 7 or 8.

Iterative Deepening Search (IDS)

- ▶ Problem: if we guess a depth limit that is too small, DLS is incomplete. If we guess one that is too large, we lose the benefit.

Iterative Deepening Search (IDS)

- ▶ Problem: if we guess a depth limit that is too small, DLS is incomplete. If we guess one that is too large, we lose the benefit.
- ▶ Idea: don't guess. Try every limit in turn.
 - ▶ run DLS with limit 0; if a solution is found, return it;
 - ▶ otherwise run DLS with limit 1; then 2; then 3; ...

Iterative Deepening Search (IDS)

- ▶ Problem: if we guess a depth limit that is too small, DLS is incomplete. If we guess one that is too large, we lose the benefit.
- ▶ Idea: don't guess. Try every limit in turn.
 - ▶ run DLS with limit 0; if a solution is found, return it;
 - ▶ otherwise run DLS with limit 1; then 2; then 3; ...
- ▶ The first solution found is at depth d , so it is a shortest solution.

Iterative Deepening Search (IDS)

- ▶ Problem: if we guess a depth limit that is too small, DLS is incomplete. If we guess one that is too large, we lose the benefit.
- ▶ Idea: don't guess. Try every limit in turn.
 - ▶ run DLS with limit 0; if a solution is found, return it;
 - ▶ otherwise run DLS with limit 1; then 2; then 3; ...
- ▶ The first solution found is at depth d , so it is a shortest solution.
- ▶ Useful when the search space is large and the depth of the solution is unknown — which is the normal situation.

General Algorithm for Iterative Deepening (IDS)

Call DLS as a subroutine:

- 1: **Input:** a start state s_0
- 2: for each state s the successors of s
- 3: a test $\text{goal}(s)$ checking whether s is a goal state
- 4:
- 5: Set $\text{DepthLimit} := 0$
- 6: **repeat**
- 7: $\text{Result} := \text{depth_limited_search}(\text{DepthLimit})$
- 8: **if** Result is a path $s_0 \dots s_k$ **then**
- 9: return $s_0 \dots s_k$ (and terminate)
- 10: **else** $\text{DepthLimit} := \text{DepthLimit} + 1$
- 11: **end if**
- 12: **until** False

General Algorithm for Iterative Deepening (IDS)

Call DLS as a subroutine:

- 1: **Input:** a start state s_0
- 2: for each state s the successors of s
- 3: a test $\text{goal}(s)$ checking whether s is a goal state
- 4:
- 5: Set $\text{DepthLimit} := 0$
- 6: **repeat**
- 7: $\text{Result} := \text{depth_limited_search}(\text{DepthLimit})$
- 8: **if** Result is a path $s_0 \dots s_k$ **then**
- 9: return $s_0 \dots s_k$ (and terminate)
- 10: **else** $\text{DepthLimit} := \text{DepthLimit} + 1$
- 11: **end if**
- 12: **until** False

Every iteration starts from scratch: the tree above the new limit is **regenerated**. This looks wasteful. The next slides show why it is not.

Properties of Iterative Deepening (IDS)

- ▶ **Complete**: if a solution exists (and b is finite) IDS finds one, because the limit eventually reaches d .

Properties of Iterative Deepening (IDS)

- ▶ **Complete**: if a solution exists (and b is finite) IDS finds one, because the limit eventually reaches d .
- ▶ **Optimal**: the solution returned is a shortest path — no solution of length $< d$ was missed, since all smaller limits failed.

Properties of Iterative Deepening (IDS)

- ▶ **Complete**: if a solution exists (and b is finite) IDS finds one, because the limit eventually reaches d .
- ▶ **Optimal**: the solution returned is a shortest path — no solution of length $< d$ was missed, since all smaller limits failed.
- ▶ **Time**: let d be the length of the shortest path to a goal state. Then in the worst case IDS will look at

$$1 + (1 + b) + (1 + b + b^2) + \cdots + (1 + b + b^2 + \cdots + b^d)$$

paths before reaching a goal state.

Properties of Iterative Deepening (IDS)

- ▶ **Complete:** if a solution exists (and b is finite) IDS finds one, because the limit eventually reaches d .
- ▶ **Optimal:** the solution returned is a shortest path — no solution of length $< d$ was missed, since all smaller limits failed.
- ▶ **Time:** let d be the length of the shortest path to a goal state. Then in the worst case IDS will look at

$$1 + (1 + b) + (1 + b + b^2) + \cdots + (1 + b + b^2 + \cdots + b^d)$$

paths before reaching a goal state.

- ▶ **Space:** $O(bd)$ — the same as DFS, since each iteration is just a DLS.

Properties of Iterative Deepening (IDS)

- ▶ **Complete**: if a solution exists (and b is finite) IDS finds one, because the limit eventually reaches d .
- ▶ **Optimal**: the solution returned is a shortest path — no solution of length $< d$ was missed, since all smaller limits failed.
- ▶ **Time**: let d be the length of the shortest path to a goal state. Then in the worst case IDS will look at

$$1 + (1 + b) + (1 + b + b^2) + \cdots + (1 + b + b^2 + \cdots + b^d)$$

paths before reaching a goal state.

- ▶ **Space**: $O(bd)$ — the same as DFS, since each iteration is just a DLS.

So IDS has the guarantees of BFS at the memory cost of DFS.

IDS versus BFS: is the regeneration expensive?

Example: $b = 10$, $d = 5$.

- ▶ BFS generates $1 + b + \dots + b^5 = 111,111$ paths and must **store** up to $b^5 = 100,000$.
- ▶ IDS generates $1 + (1 + b) + \dots + (1 + \dots + b^5) = 123,456$ paths and stores only about $b \cdot d = 50$.
- ▶ Cost: 11% more work. Benefit: $2000\times$ less memory.

IDS versus BFS: is the regeneration expensive?

Example: $b = 10$, $d = 5$.

- ▶ BFS generates $1 + b + \dots + b^5 = 111,111$ paths and must **store** up to $b^5 = 100,000$.
- ▶ IDS generates $1 + (1 + b) + \dots + (1 + \dots + b^5) = 123,456$ paths and stores only about $b \cdot d = 50$.
- ▶ Cost: 11% more work. Benefit: $2000\times$ less memory.

Why so cheap? Because the last level dominates: it contains more nodes than **all** the levels above it put together. Redoing everything above it is almost free.

Where IDS is naturally used

- ▶ **IDA*** (iterative deepening A^* , covered soon) is the standard optimal solver for the 15-puzzle and for Rubik's cube: it iterates on a *cost* bound rather than a depth bound.

Where IDS is naturally used

- ▶ **IDA*** (iterative deepening A^* , covered soon) is the standard optimal solver for the 15-puzzle and for Rubik's cube: it iterates on a *cost* bound rather than a depth bound.
- ▶ A web crawler that must find the shortest click-path from a page to another: you can't hold a BFS frontier of the web in RAM, but you can hold one root-to-page path.

Where IDS is naturally used

- ▶ **IDA*** (iterative deepening A^* , covered soon) is the standard optimal solver for the 15-puzzle and for Rubik's cube: it iterates on a *cost* bound rather than a depth bound.
- ▶ A web crawler that must find the shortest click-path from a page to another: you can't hold a BFS frontier of the web in RAM, but you can hold one root-to-page path.

Whenever memory — not time — is the binding constraint, iterative deepening is the first thing to try.

Algorithmic Improvements

Speeding things up:
avoiding repeated states, and searching from both
ends.

Repeated states: how bad is it?

A robot on a grid can move N, E, S, W, so $b = 4$.

- ▶ Number of **paths** of length d from the start: up to 4^d .
- ▶ Number of **states** reachable in d steps: only $2d^2 + 2d + 1$.

Repeated states: how bad is it?

A robot on a grid can move N, E, S, W, so $b = 4$.

- ▶ Number of **paths** of length d from the start: up to 4^d .
- ▶ Number of **states** reachable in d steps: only $2d^2 + 2d + 1$.

d	paths of length d	states within d steps
5	1,024	61
10	$\approx 1.0 \times 10^6$	221
20	$\approx 1.1 \times 10^{12}$	841

Repeated states: how bad is it?

A robot on a grid can move N, E, S, W, so $b = 4$.

- ▶ Number of **paths** of length d from the start: up to 4^d .
- ▶ Number of **states** reachable in d steps: only $2d^2 + 2d + 1$.

d	paths of length d	states within d steps
5	1,024	61
10	$\approx 1.0 \times 10^6$	221
20	$\approx 1.1 \times 10^{12}$	841

The search tree is exponential; the state *graph* is quadratic.
Almost all the work of a tree search is re-exploring states it has already seen.

Avoiding Repeated States

Three options, in order of increasing effectiveness *and* increasing overhead:

1. Do not return to the state you have just come from.

Avoiding Repeated States

Three options, in order of increasing effectiveness *and* increasing overhead:

1. Do not return to the state you have just come from.
2. Do not create paths with a **cycle** in them (no state occurs twice on the same path).

Avoiding Repeated States

Three options, in order of increasing effectiveness *and* increasing overhead:

1. Do not return to the state you have just come from.
2. Do not create paths with a **cycle** in them (no state occurs twice on the same path).
3. Do not create paths containing **any** state that occurred in a path created earlier — set of **reached** states.
This turns tree search into **graph search**.

Avoiding Repeated States

Three options, in order of increasing effectiveness *and* increasing overhead:

1. Do not return to the state you have just come from.
2. Do not create paths with a **cycle** in them (no state occurs twice on the same path).
3. Do not create paths containing **any** state that occurred in a path created earlier — set of **reached** states.
This turns tree search into **graph search**.

The trade-off is real: option 3 makes BFS on the grid above polynomial instead of exponential, but it also destroys the memory advantage of DFS and IDS, which is a main reason we were using them. So: BFS is usually run as graph search (option 3); DFS and IDS usually use option 2.

Bi-directional Search

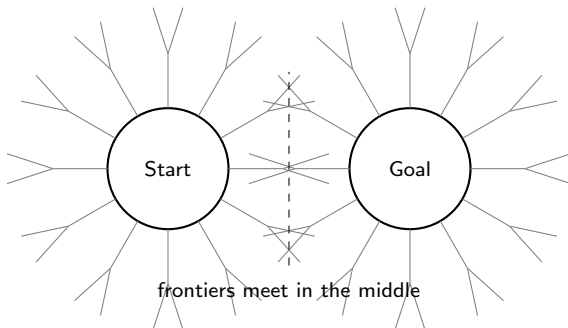
- ▶ Suppose we search from **the goal state backwards** as well as from the **initial state forwards**.

Bi-directional Search

- ▶ Suppose we search from **the goal state backwards** as well as from the **initial state forwards**.
- ▶ This involves determining **predecessor** states of the goal, then predecessors of those, ...

Bi-directional Search

- ▶ Suppose we search from **the goal state backwards** as well as from the **initial state forwards**.
- ▶ This involves determining **predecessor** states of the goal, then predecessors of those, ...
- ▶ Stop when the two frontiers **meet**: a state reached from both ends gives a complete path (forwards to that state, then backwards to the goal).



Bi-directional Search: good properties

- ▶ Often much more efficient: instead of one search to depth d , we do two searches to depth $d/2$.
- ▶ $2b^{d/2}$ is dramatically smaller than b^d .

Bi-directional Search: good properties

- ▶ Often much more efficient: instead of one search to depth d , we do two searches to depth $d/2$.
- ▶ $2b^{d/2}$ is dramatically smaller than b^d .
- ▶ Example: $b = 10$, $d = 6$.
 - ▶ BFS examines $1 + b + \dots + b^6 = 1,111,111$ paths.
 - ▶ Bi-directional BFS examines $2 \times (1 + b + b^2 + b^3) = 2,222$ paths.

Bi-directional Search: good properties

- ▶ Often much more efficient: instead of one search to depth d , we do two searches to depth $d/2$.
- ▶ $2b^{d/2}$ is dramatically smaller than b^d .
- ▶ Example: $b = 10$, $d = 6$.
 - ▶ BFS examines $1 + b + \dots + b^6 = 1,111,111$ paths.
 - ▶ Bi-directional BFS examines $2 \times (1 + b + b^2 + b^3) = 2,222$ paths.
- ▶ The memory requirement drops in the same way: $b^{d/2}$ per direction rather than b^d .

Bi-directional Search: good properties

- ▶ Often much more efficient: instead of one search to depth d , we do two searches to depth $d/2$.
- ▶ $2b^{d/2}$ is dramatically smaller than b^d .
- ▶ Example: $b = 10$, $d = 6$.
 - ▶ BFS examines $1 + b + \dots + b^6 = 1,111,111$ paths.
 - ▶ Bi-directional BFS examines $2 \times (1 + b + b^2 + b^3) = 2,222$ paths.
- ▶ The memory requirement drops in the same way: $b^{d/2}$ per direction rather than b^d .
- ▶ Different strategies can be used in the two directions.

Bi-directional Search: problematic properties

- ▶ We must be able to **generate predecessors** of a state. Easy on a map (roads run both ways); hard when actions destroy information (e.g. “delete the file”).

Bi-directional Search: problematic properties

- ▶ We must be able to **generate predecessors** of a state. Easy on a map (roads run both ways); hard when actions destroy information (e.g. “delete the file”).
- ▶ We need an **explicit goal state**. If the goal is described by a property (“a checkmate position”), there may be billions of goal states to search back from.

Bi-directional Search: problematic properties

- ▶ We must be able to **generate predecessors** of a state. Easy on a map (roads run both ways); hard when actions destroy information (e.g. “delete the file”).
- ▶ We need an **explicit goal state**. If the goal is described by a property (“a checkmate position”), there may be billions of goal states to search back from.
- ▶ We need an efficient test of whether a newly generated state appears in the *other* frontier — in practice a hash table.

Bi-directional Search: problematic properties

- ▶ We must be able to **generate predecessors** of a state. Easy on a map (roads run both ways); hard when actions destroy information (e.g. “delete the file”).
- ▶ We need an **explicit goal state**. If the goal is described by a property (“a checkmate position”), there may be billions of goal states to search back from.
- ▶ We need an efficient test of whether a newly generated state appears in the *other* frontier — in practice a hash table.
- ▶ **Optimality needs care**: the *first* meeting point of the two frontiers is not always on a shortest path. A correct implementation must finish the current level and compare all meeting points.

Bi-directional Search: problematic properties

- ▶ We must be able to **generate predecessors** of a state. Easy on a map (roads run both ways); hard when actions destroy information (e.g. “delete the file”).
- ▶ We need an **explicit goal state**. If the goal is described by a property (“a checkmate position”), there may be billions of goal states to search back from.
- ▶ We need an efficient test of whether a newly generated state appears in the *other* frontier — in practice a hash table.
- ▶ **Optimality needs care**: the *first* meeting point of the two frontiers is not always on a shortest path. A correct implementation must finish the current level and compare all meeting points.
- ▶ Memory is still $O(b^{d/2})$ per direction: better, but still exponential. For large d this remains impractical.

Everything at a glance

	Complete?	Optimal?	Time	Space
Breadth-first (BFS)	yes	yes	$O(b^d)$	$O(b^d)$
Depth-first (DFS)	no	no	$O(b^m)$	$O(b\ m)$
Depth-limited (DLS)	if $d \leq \ell$	no	$O(b^\ell)$	$O(b\ \ell)$
Iterative deepening (IDS)	yes	yes	$O(b^d)$	$O(b\ d)$
Bi-directional (BFS both)	yes	yes	$O(b^{d/2})$	$O(b^{d/2})$

Read the table by column: **IDS is the sweet spot** for uninformed search — BFS's guarantees, DFS's memory. Bi-directional search beats it on time and space, but only when you can run the state space backwards from an explicit goal.

Interactive demo

An in-browser visualiser for BFS, DFS, DLS, IDS and bi-directional search on a maze is on canvas ([download](#)).

Watch the two counters as you switch algorithm:

- ▶ **paths generated** (time), and
- ▶ **largest frontier** (space).

Interactive demo

An in-browser visualiser for BFS, DFS, DLS, IDS and bi-directional search on a maze is on canvas ([download](#)).

Watch the two counters as you switch algorithm:

- ▶ **paths generated** (time), and
- ▶ **largest frontier** (space).

Try to reproduce, by hand, the numbers on the previous table: run BFS and IDS on the same maze, and check that IDS generates only a little more, while its frontier stays tiny.

Quiz (2 minutes)

You are searching for a route in a state space with $b = 10$ and $d = 8$. Your machine can store 10^6 paths.

1. Can you use BFS?
2. Can you use IDS?
3. You now learn that every action is reversible and the goal is a single known state. What do you do?

Quiz (2 minutes)

You are searching for a route in a state space with $b = 10$ and $d = 8$. Your machine can store 10^6 paths.

1. Can you use BFS?
2. Can you use IDS?
3. You now learn that every action is reversible and the goal is a single known state. What do you do?

Answers: (1) no — BFS needs about 10^8 paths in memory; (2) yes — IDS needs about $b \cdot d = 80$, at $\approx 11\%$ extra time; (3) bi-directional search: two searches to depth 4, about $2 \cdot 10^4$ paths in memory and far less time.

Summary

- ▶ More advanced uninformed search techniques:
 - ▶ depth limited search — DFS made to terminate;
 - ▶ iterative deepening — DFS made complete and optimal;
 - ▶ avoiding repeated states — three levels, three different memory bills;
 - ▶ bi-directional search — b^d becomes $2b^{d/2}$, if you can search backwards.

Summary

- ▶ More advanced uninformed search techniques:
 - ▶ depth limited search — DFS made to terminate;
 - ▶ iterative deepening — DFS made complete and optimal;
 - ▶ avoiding repeated states — three levels, three different memory bills;
 - ▶ bi-directional search — b^d becomes $2b^{d/2}$, if you can search backwards.
- ▶ These improve substantially on plain BFS and DFS.

Summary

- ▶ More advanced uninformed search techniques:
 - ▶ depth limited search — DFS made to terminate;
 - ▶ iterative deepening — DFS made complete and optimal;
 - ▶ avoiding repeated states — three levels, three different memory bills;
 - ▶ bi-directional search — b^d becomes $2b^{d/2}$, if you can search backwards.
- ▶ These improve substantially on plain BFS and DFS.
- ▶ But they are all still **uninformed**: they know nothing about which direction the goal lies in, and remain exponential in d . Realistic problems need **heuristics** — the topic of the next section.